

SQL MASTERY

Week 6 — JOIN & SubQuery

BBK1LAB3 — Sistem Basis Data
Program Studi S1 Sistem Informasi — Fakultas Rekayasa Industri

01	SubQuery	Nested queries, inline views
02	SubQuery Rules	Syntax rules & restrictions
03	ANY / ALL / EXISTS	Conditional subquery keywords
04	INNER JOIN	Combine matching rows
05	LEFT / RIGHT JOIN	Include non-matching rows
06	NATURAL JOIN	Auto-match on same columns
07	FULL JOIN	All rows from both tables
08	Summary & Cheatsheet	Quick reference guide

SubQuery

Query yang berada di dalam query lain (Nested Query / SubSelect)

Apa itu SubQuery?

SubQuery adalah bentuk query yang tertanam di dalam query SQL lain.

Disebut juga: Nested Query atau SubSelect.

Memungkinkan adanya statement SELECT di dalam SQL lainnya (SELECT, INSERT, UPDATE, DELETE).

Kegunaan SubQuery

- > Meng-copy data dari satu tabel ke tabel lain
- > Menerima data dari inline view
- > Mengambil data dari tabel lain untuk di-UPDATE ke tabel tujuan
- > Menghapus baris dari satu tabel berdasarkan kondisi dari tabel lain

Aturan SubQuery

- Subquery HARUS tertutup dalam tanda kurung ()
- Klausa ORDER BY TIDAK BOLEH digunakan di dalam subquery (boleh di luar)
- SELECT di subquery harus berisi SATU nama kolom/ekspresi (kecuali EXISTS)
- Nama kolom di subquery mengacu ke tabel di klausa FROM query luar
- Subquery TIDAK BOLEH digunakan sebagai operan dalam ekspresi

SQL — Sintaks Dasar SubQuery

```
SELECT kolom1
FROM   tabel_luar
WHERE  kolom2 = (SELECT kolom2
                  FROM   tabel_dalam
                  WHERE  kondisi);
```


ANY, ALL & EXISTS

Keyword kondisional pada SubQuery

ANY — Satu Saja
Bernilai TRUE jika setidaknya

SATU nilai dari subquery
memenuhi kondisi.

```
WHERE nilai > ANY (...)
```

ALL — Semua Nilai
Bernilai TRUE hanya jika

SEMUA nilai dari subquery
memenuhi kondisi.

```
WHERE nilai > ALL (...)
```

EXISTS — Ada
Bernilai TRUE jika subquery

menghasilkan setidaknya
SATU baris hasil.

```
WHERE EXISTS (SELECT ...)
```

NOT EXISTS — Tidak Ada
Kebalikan EXISTS

TRUE jika subquery TIDAK
menghasilkan baris apapun.

```
WHERE NOT EXISTS (...)
```

Contoh Penggunaan

ANY — Nilai UAS di atas rata-rata salah satu mahasiswa

```
SELECT nim, kode_mk, uas
FROM   tb_nilai
WHERE  uas > ANY (SELECT uas FROM tb_nilai WHERE nim = '1301140017');
```

EXISTS — Mahasiswa yang mengikuti ujian

```
SELECT nama
FROM   tb_mahasiswa m
WHERE  EXISTS (SELECT 1 FROM tb_nilai n
                WHERE  n.nim = m.nim);
```

NOT EXISTS — Mahasiswa yang TIDAK mengikuti ujian

```
SELECT nama
FROM   tb_mahasiswa m
WHERE  NOT EXISTS (SELECT 1 FROM tb_nilai n
                   WHERE  n.nim = m.nim);
```


Contoh SubQuery

Query nyata berdasarkan tabel tb_mahasiswa & tb_nilai

Ambil nilai UTS & UAS dari mahasiswa bernama Tiara Lestari

```
SELECT uts, uas
FROM   tb_nilai
WHERE  nim = (SELECT nim
              FROM   tb_mahasiswa
              WHERE  nama = 'Tiara Lestari');
```

Ambil NIM, nilai UTS & UAS mahasiswa yang tinggal di Jl. Margacinta

```
SELECT nim, uts, uas
FROM   tb_nilai
WHERE  nim IN (SELECT nim
              FROM   tb_mahasiswa
              WHERE  alamat LIKE '%Margacinta%');
```

Tampilkan NIM, kode_mk dari tb_nilai yang nilai UAS-nya di atas 70

```
SELECT nim, kode_mk
FROM   tb_nilai
WHERE  uas > 70;
```

NIM, kode_mk, UAS — nilai UAS di atas rata-rata

```
SELECT nim, kode_mk, uas
FROM   tb_nilai
WHERE  uas > (SELECT AVG(uas) FROM tb_nilai);
```

Nama MK yang SKS-nya lebih besar dari MK Algoritma dan Pemrograman

```
SELECT nama_mk
FROM   tb_mk
WHERE  sks > (SELECT sks FROM tb_mk
              WHERE  nama_mk = 'Algoritma dan Pemrograman');
```


SQL JOIN

Menggabungkan data dari dua atau lebih tabel

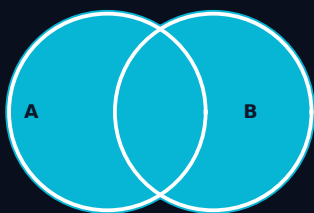
Mengapa JOIN?

Data relasional tersebar di banyak tabel.

JOIN memungkinkan kita menampilkan data dari MULTIPLE TABEL dalam satu query.

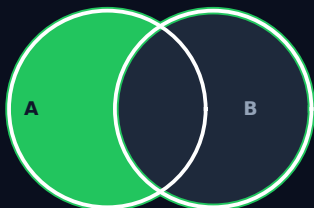
Relasi antar tabel dibuat melalui kolom yang sama (Foreign Key -> Primary Key).

Tipe-tipe JOIN di MySQL



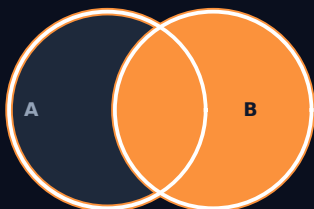
INNER JOIN

Hanya baris yang COCOK di kedua tabel. INNER JOIN = CROSS JOIN = JOIN.



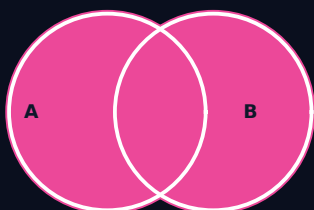
LEFT JOIN

Semua baris tabel KIRI + baris yang cocok dari kanan. NULL jika tidak ada.



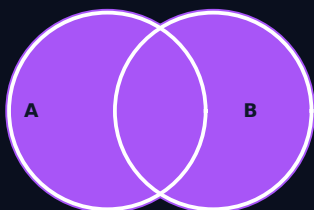
RIGHT JOIN

Semua baris tabel KANAN + baris yang cocok dari kiri. NULL jika tidak ada.



FULL JOIN

SEMUA baris dari kedua tabel. NULL untuk yang tidak cocok.



NATURAL JOIN

Auto-join berdasarkan kolom dengan NAMA SAMA. Tidak perlu ON/USING.

INNER JOIN

Menampilkan baris yang cocok di kedua tabel

Cara Kerja INNER JOIN

Hanya baris yang memiliki nilai SAMA pada kolom yang di-JOIN yang akan tampil.

Baris yang tidak memiliki pasangan di tabel lain TIDAK akan ditampilkan.

Di MySQL: INNER JOIN = CROSS JOIN = JOIN (dengan klausa ON)

INNER JOIN — menggunakan ON

```
SELECT *  
FROM   tb_dosen  
INNER JOIN tb_mk ON tb_dosen.kode_dsn = tb_mk.kode_dsn;
```

INNER JOIN — menggunakan USING (hapus redundansi kolom)

```
SELECT *  
FROM   tb_dosen  
INNER JOIN tb_mk USING (kode_dsn);
```

Alternatif: Koma + WHERE (sama dengan INNER JOIN)

```
SELECT *  
FROM   tb_dosen, tb_mk  
WHERE  tb_dosen.kode_dsn = tb_mk.kode_dsn;
```

Catatan Penting INNER JOIN

ON : kolom duplikat TETAP muncul (2 kolom kode_dsn)

USING : kolom duplikat DIHAPUS otomatis (1 kolom kode_dsn)

Koma+WHERE : ekuivalen dengan INNER JOIN, urutan baris ikut operand kanan

LEFT & RIGHT JOIN

Menampilkan data termasuk baris yang tidak memiliki pasangan

LEFT JOIN

LEFT JOIN — Semua data tabel KIRI dipertahankan

Semua baris dari tabel KIRI ditampilkan meskipun tidak ada pasangan di tabel kanan.

Kolom dari tabel kanan akan bernilai NULL jika tidak ada data yang cocok.

Urutan data dan urutan baris mengikuti operand KIRI.

LEFT JOIN — tb_dosen ke tb_mk

```
SELECT *  
  
FROM   tb_dosen  
  
LEFT JOIN tb_mk ON tb_dosen.kode_dsn = tb_mk.kode_dsn;  
  
-- atau dengan USING:  
SELECT * FROM tb_dosen LEFT JOIN tb_mk USING (kode_dsn);
```

RIGHT JOIN

RIGHT JOIN — Semua data tabel KANAN dipertahankan

Semua baris dari tabel KANAN ditampilkan meskipun tidak ada pasangan di tabel kiri.

Kolom dari tabel kiri akan bernilai NULL jika tidak ada data yang cocok.

Urutan data dan urutan baris mengikuti operand KANAN.

RIGHT JOIN — tb_dosen ke tb_mk

```
SELECT *  
  
FROM   tb_dosen  
  
RIGHT JOIN tb_mk ON tb_dosen.kode_dsn = tb_mk.kode_dsn;  
  
-- atau dengan USING:  
SELECT * FROM tb_dosen RIGHT JOIN tb_mk USING (kode_dsn);
```


NATURAL JOIN

Join otomatis berdasarkan kolom dengan nama yang sama

Keistimewaan NATURAL JOIN

Tidak perlu menentukan ON, WHERE, atau USING — otomatis cari kolom bernama sama.

Kolom yang sama HANYA DITAMPILKAN SATU KALI (tidak redundan).

Hanya baris dengan nilai SAMA pada kolom yang sama yang muncul.

NATURAL JOIN

```
SELECT *
FROM   tb_dosen
NATURAL JOIN tb_mk;

-- Kolom kode_dsn hanya muncul sekali
-- Hanya data yang cocok yang ditampilkan
```

NATURAL LEFT JOIN

Urutan KOLOM mengikuti operand kiri.

DATA mengikuti operand kanan.

```
SELECT * FROM tb_dosen
NATURAL LEFT JOIN tb_mk;
```

NATURAL RIGHT JOIN

Urutan KOLOM mengikuti operand kanan.

DATA mengikuti operand kiri.

```
SELECT * FROM tb_dosen
NATURAL RIGHT JOIN tb_mk;
```

FULL JOIN (Simulasi di MySQL)

MySQL tidak punya FULL JOIN langsung — gunakan UNION

Gabungkan LEFT JOIN + RIGHT JOIN dengan UNION untuk mendapatkan FULL OUTER JOIN.

FULL JOIN menggunakan UNION

```
SELECT * FROM tb_dosen
LEFT JOIN tb_mk ON tb_dosen.kode_dsn = tb_mk.kode_dsn
UNION
SELECT * FROM tb_dosen
RIGHT JOIN tb_mk ON tb_dosen.kode_dsn = tb_mk.kode_dsn;
```


Quick Reference

Rangkuman JOIN & SubQuery untuk belajar cepat

JOIN Type	Keyword	Hasil	Urutan
INNER JOIN	INNER JOIN ... ON/USING	Cocok di keduanya	Ikut operand kanan
LEFT JOIN	LEFT JOIN ... ON/USING	Semua kiri + cocok	Ikut operand kiri
RIGHT JOIN	RIGHT JOIN ... ON/USING	Semua kanan + cocok	Ikut operand kanan
NATURAL	NATURAL JOIN	Cocok, kolom unik	Ikut kolom kiri
NAT LEFT	NATURAL LEFT JOIN	Kolom kiri, data kanan	Kolom kiri
NAT RIGHT	NATURAL RIGHT JOIN	Kolom kanan, data kiri	Kolom kanan
FULL JOIN	LEFT UNION RIGHT	Semua baris	Keduanya

SubQuery vs JOIN — Kapan Pakai Apa?

Gunakan SubQuery jika:

- Butuh hasil agregat (AVG, MAX, MIN)
- Filter berdasarkan kolom tabel lain
- Cek keberadaan data (EXISTS)
- Logika bersarang yang kompleks

Gunakan JOIN jika:

- Butuh kolom dari BANYAK tabel
- Perlu tampilkan data gabungan
- Performance lebih diutamakan
- Relasi jelas antar tabel

Tips Belajar

1. Pahami struktur tabel terlebih dahulu (kolom, primary key, foreign key)
2. Mulai dari INNER JOIN, lalu LEFT/RIGHT, baru NATURAL dan subquery
3. Gunakan alias tabel untuk query panjang: tb_mahasiswa m, tb_nilai n
4. Selalu test dengan data kecil sebelum query di database besar
5. EXPLAIN query untuk memahami performance di MySQL

